



■ 소개

"AI의 작업을 통제하고 검증해, 생성물을 이해하고 설명하며 책임질 수 있는 결과로 만드는 AI-native 개발자."

업무의 반복·탐색·문서 처리·검토 과정에서 생기는 문제를 해결하기 위해 필요한 AI 기술을 선택하고 서비스로 구현해왔습니다.

기술 자체보다 왜 이 기술이 필요한지, 어떤 한계가 있는지, 사람이 어디서 개입하고 검증해야 하는지를 기준으로 설계합니다.

■ 기술

언어	Python · Java · SQL
AI / Agent	LangGraph · LangChain4j · MCP · Agent Skills · Claude Code · Codex
검색 / 데이터	Qdrant · Hybrid Search · Embedding · MySQL · SQLite
LLM 실행	Ollama · AWS SageMaker JumpStart
개발 도구	Git · GitHub

■ 주요 프로젝트

FOWOCO | 외국인 근로자 HR 업무 지원 서비스 [\[GitHub ↗\]](#)

2026.06 - 2026.08 (8인 팀 · 완료)

담당: Language Assistant 서브그래프 / HWPX MCP 책임 분리 / 8인 온라인 협업 조율

Python LangGraph Qdrant BGE-M3 MCP FastAPI

도메인 특화 Hybrid Search & Re-ranking RAG 파이프라인 구현 및 검증

[문제] 고용허가제(EPS) 행정 서식과 외국인 근로자 질의 간 어휘 불일치로 인한 검색 누락 발생

[판단·구현] 3개 고정 검색 관점을 수렴하고, Dense(BGE-M3)+Sparse 결합 Hybrid Search 및 RRF, Re-ranking 흐름 구현

[구현 결과] BGE-M3 dense/sparse → RRF → Re-ranking 검색 흐름 구축 (후보 문서 30개 → 상위 5개 선별)

[검증·회고] 정량 벤치마크를 통해 단일 Dense(99%) 대비 3중 분기 지연 시간(60→390ms) 트레이드오프를 확인하고 단순화 방향 도출

워크플로우 조율과 문서 처리의 책임 분리 (MCP 독립화)

[문제] LangGraph 에이전트 내부에 HWPX 파싱/편집 로직이 결합되어 그래프 복잡도 증가 및 에러 전파 위험

[판단·구현] 문서 처리 로직을 별도 MCP(Model Context Protocol) 서버로 분리해 Agent Workflow와 독립적으로 관리할 수 있는 Tool 경계를 구현

[구현 결과] 문서 처리 실패가 Main Workflow로 직접 전파되지 않도록 책임 경계를 분리하고, 독립적으로 관리·재사용할 수 있는 구조로 전환

8인 온라인 협업 조율 및 3단계 의사결정 구조 정립

[판단·구현] 8인 전원 온라인 협업 환경에서 진행자를 보완해 의견을 정리하고, [2개 조 논의 → 조장 정리 → 전체 합의] 구조로 의사결정 흐름을 개선

[변화] 모든 구성원이 의견을 낼 수 있는 참여 구조를 확보하고, 회의의 논점 분산과 갈등을 줄여 의사결정을 원활하게 함

HWPX Document Plugin | 에이전트 기반 한글 문서 편집 지원 [\[GitHub ↗\]](#)

2026.07 - 현재 (2인 팀 · 개발 중)

담당: MCP 서버 전반 설계 및 구현 / Agent Skills 설계 및 개발

Python MCP Agent Skills HWPX(XML) Vision

XML 구조·렌더링 이미지 듀얼 분석 및 잘못된 필드 매핑 위험 제어

[문제] HWPX의 XML만으로는 라벨·빈칸·표 구조 사이의 의미적 관계와 실제 입력 위치를 충분히 파악하기 어려워 잘못된 필드 매핑 위험 존재

[판단·구현] XML 노드 구조와 렌더링 이미지를 함께 분석하는 매핑 방식 설계. [분석 → 인터뷰 확인 → 실행 → 검증] 단계를 Agent Skills로 모듈화하여 공란·수기 여부를 확정 후 반영

[통제 방식] 불확실한 필드 매핑을 사용자 확인 대상으로 분리해 AI가 임의로 입력하지 않도록 통제

FOWOCO Node → MCP 책임 분리 → 독립 Agent Plugin으로의 구조 진화

[진화 과정] FOWOCO 내부 노드로 시작했던 HWPX 처리 로직을 MCP로 1차 분리해 결합도를 낮췄고, 이를 특정 FOWOCO Workflow에 종속되지 않는 독립 Agent Plugin으로 팀과 함께 확장 중

■ 주요 프로젝트 (이어서)

OwnHands | AI 코딩 작업의 근거(Evidence) 기반 검토 엔진 [\[GitHub 7\]](#)

2026.09 - 현재 (개인 프로젝트 · 개발 중)

담당: 제품 아키텍처 설계 / Evidence Core 및 Issue 기반 검증 흐름 개발

Python

SQLite

MCP

Agent Skills

완료 조건(Spec) 기반 검증 라이프사이클 설계 및 개발

[문제] AI 코딩 도구가 생성한 diff만으로는 사이드 이펙트나 실제 스펙 충족 여부를 사람이 빠르게 확인하기 어려움

[판단·구현] [Issue → Verification Spec → Baseline → 구현 → Review → Human Decision]으로 구현 전 완료 조건과 검증 기준을 고정하는 Lifecycle 파이프라인 설계·개발

[구현 상태] 사전 정의한 Spec과 실제 변경·테스트 상태를 연결해 사람이 작업 결과의 수용 여부를 판단할 수 있는 검토 구조를 설계

Claim-Evidence 분리 기반 회귀 판정 코어 엔진 구현

[판단·구현] AI의 '주장(Claim)'과 실제 실행된 '테스트 로그(Evidence)'를 분리하여 SQLite Event Store에 저장하고, 요구사항별 검증 상태와 Human Decision을 연결하는 회귀 판정 엔진 개발

[구현 상태] AI의 Claim과 실행 Evidence를 분리해 저장하고, 사람이 실제 근거를 확인한 뒤 최종 판단하도록 구조화

LLM Gateway Service | 다중 LLM 선택 및 호출 통합 게이트웨이 [\[GitHub 7\]](#)

2025.04 - 2025.10 (2인 팀 · 완료)

담당: LLM Provider 연동 / 다기준 모델 우선순위 계산 로직 / 모델 호스팅 환경 구성

Java

Spring Boot

LangChain4j

WebSocket

MySQL

Ollama

다기준 모델 우선순위 계산 로직 구현

[문제] 모델별 특성이 달라 사용자가 목적에 맞는 모델을 비교·선택하기 어렵고, 초기 임계값 방식에서는 조건이 겹칠수록 후보가 사라짐

[판단·구현] 비용·속도·성능·Context Length 4대 기준을 Min-Max 정규화하고 사용자 가중치를 반영해 모델 우선순위를 계산하는 상대 평가 라우터 구현

오픈소스 LLM 호스팅 및 통신 아키텍처 회고

[판단·구현] AWS SageMaker JumpStart로 전체 흐름의 구현 가능성을 빠르게 POC한 뒤, 베타 단계에서는 보유 GPU 서버에 오픈소스 LLM(Ollama)을 호스팅하고 LangChain4j로 연동

[회고] 당시에는 LLM과 실시간 상호작용하려면 양방향 통신이 필요하다고 판단해 WebSocket을 선택했다. 구현 후 핵심 요구가 서버의 토큰 스트리밍임을 이해했고, 현재라면 REST + SSE를 우선 검토한다.

■ 학력 및 교육

학력	교육 이수
상명대학교 소프트웨어학과 학사 졸업 학점 3.96 / 4.5	2020.03 - 2026.02
	KT그룹 AX전략 기획 및 멀티에이전트 서비스 개발 과정 2026.09.07 - 현재 10주 과정 (진행 중)
	KT AIVLE School 9기 AI Developer 2026.04 - 2026.09.03 수료

■ 자격증

정보처리기사 한국산업인력공단 국가기술자격 (SW 전반 공인)	2025.09	ISTQB CTFL (국제 공인 SW 테스트)	2026.02
		KSTQB / ISTQB 테스트 원칙 및 테스트 설계 기법	
AICE Associate (인공지능 활용능력) KT / 한국경제신문 주관 AI 실무 응용 자격	2026.04	SQLD (SQL 개발자)	2025.06
		한국데이터산업진흥원 데이터 모델링 및 SQL 튜닝	
ADsP (데이터분석 준전문가) 한국데이터산업진흥원 데이터 통계 분석 및 기획	2025.06		